pgvector: Hybrid Search

Spiegazioni semplici, esempi SQL e limiti pratici

Che cosa significa

La ricerca ibrida combina ricerca vettoriale e ricerca testuale. La prima confronta embedding; la seconda usa i termini del testo, per esempio tramite il full-text search di PostgreSQL. Le due classifiche vengono unite in una risposta finale.

Perché combinarle

La ricerca vettoriale può recuperare parafrasi e formulazioni semanticamente vicine. La ricerca testuale aggiunge un segnale lessicale. Per “ridurre i consumi”, una ricerca può valorizzare “efficienza energetica” mentre l’altra favorisce documenti con i termini cercati. È un esempio intuitivo: i risultati effettivi dipendono da modello e documenti.

Non confonderla con il filtering

WHERE lingua = 'it' limita i documenti ammessi, ma non aggiunge una seconda classifica di rilevanza. Una ricerca vettoriale con un filtro non è, per questo solo fatto, una ricerca ibrida. I filtri possono essere applicati a entrambi i rami della ricerca ibrida.

Preparare il full-text search

Esempio su una tabella documenti con colonne id, titolo, testo ed embedding. La colonna generata usa la stessa configurazione linguistica impiegata nelle query. Il full-text italiano elabora i termini: non è una semplice ricerca di sottostringhe né una garanzia di corrispondenza letterale.

```
ALTER TABLE documenti ADD COLUMN textsearch tsvector
GENERATED ALWAYS AS (
  to_tsvector('italian', coalesce(testo, ''))
) STORED;

CREATE INDEX documenti_fts
ON documenti USING gin (textsearch);
```

Ramo vettoriale

Genera l’embedding della domanda con un modello compatibile con quello dei documenti. Recupera una lista di candidati ordinata per distanza. Il numero di candidati è un parametro da valutare, non deve necessariamente coincidere con il numero di risposte finali.

```
SELECT id, titolo
FROM documenti
ORDER BY embedding <=> $1::vector
LIMIT 50;
```

Ramo testuale

Recupera una seconda lista ordinata con un punteggio testuale. Se sono richieste corrispondenze esatte di codici o identificativi, verifica la tokenizzazione: il full-text può non essere sufficiente senza una ricerca dedicata.

```
SELECT d.id, d.titolo
FROM documenti AS d
CROSS JOIN plainto_tsquery('italian', $2) AS q
WHERE d.textsearch @@ q
```

```
ORDER BY ts_rank_cd(d.textsearch, q) DESC
LIMIT 50;
```

Fusione con RRF

Reciprocal Rank Fusion combina le posizioni, non i valori grezzi: score(d) = somma di 1/(k + posizione_ramo(d)). Le posizioni partono da 1; un documento assente da un ramo non contribuisce in quel ramo. k è una costante positiva da scegliere. Si ordina il risultato per score decrescente. Questo evita di sommare direttamente distanze coseno e punteggi testuali non confrontabili.

Reranking e valutazione

In alternativa o dopo la fusione, un cross-encoder può rivalutare le coppie domanda-documento per riordinarle. È un passaggio esterno al semplice confronto tra embedding e aggiunge costo. Valuta separatamente recupero dei candidati, ordine finale, latenza e qualità su domande reali.

Fonte e ambito

Sintesi didattica degli argomenti discussi, basata sulla documentazione ufficiale di pgvector. I parametri mostrati non sostituiscono benchmark sui propri dati. Le funzioni disponibili dipendono dalla versione installata.

https://github.com/pgvector/pgvector/blob/master/README.md